



Agentic AI Engineering Internship , 2026

WEEK 2

Prompt Engineering & Tool Calling

Monday 29 June – Friday 3 July 2026

Mastering the language of AI: structured prompts and callable tools

Week Overview

Week 2 builds on the foundations of Week 1 by introducing advanced prompt engineering patterns and structured tool calling , the two pillars that allow an LLM to move from passive conversation to reliable, controllable action.

Detail	Information
Week Dates	Monday 29 June – Friday 3 July 2026
Theme	Prompt Engineering & Tool Calling
Primary Stack	Python · LangChain · Groq API · Pydantic v2 · httpx
Total Commitment	20 hours (4 hours/day · 5 days)
Deliverable Due Friday	1 Intermediate Project + 1 Production Project (choose one of each)

Learning Objectives

Master advanced prompt engineering patterns (CoT, ToT, few-shot, meta-prompting)
Implement function/tool calling with structured outputs using Pydantic
Build agents that call external APIs, databases, and code execution engines
Handle tool call errors, retries, and fallbacks gracefully
Design tool schemas following OpenAI/Anthropic best practices
Build a multi-tool agent pipeline end-to-end

Topics & Subtopics

Topic	Subtopics
Advanced Prompt Engineering	Chain-of-thought (CoT), Tree-of-thought (ToT), self-consistency, meta-prompting, prompt chaining, negative prompting
Structured Outputs	Pydantic v2 models, JSON mode, function calling specification, output parsers, validation
Tool/Function Calling	Tool schemas (OpenAI format), ToolMessage, tool selection logic, parallel tool calls, tool chaining
External Tool Integration	HTTP tools (httpx), code execution, file operations, database queries, web search integration
Error Handling	Retry logic, fallback chains, graceful degradation, error types in LLM APIs
LangChain Tools	Tool decorator, BaseTool class, StructuredTool, tool binding to ChatModels

Daily Breakdown

Each day is structured around a core learning block followed by applied, hands-on practice. Complete each day's practice before moving to the next , concepts compound throughout the week.

Day	Focus	Core Learning	Applied Practice
Mon	Advanced Prompting	Study CoT, ToT, self-consistency papers. Read Anthropic Prompt Engineering Guide in full. Practice 10 prompt rewrites.	Lab: Build a CoT prompting pipeline. Compare answers with/without CoT on 10 math and logic problems. Document improvements.
Tue	Structured Outputs	Study Pydantic v2: validators, model_config, field aliases. Build 5 complex Pydantic models for AI output schemas.	Implement structured output extraction using LangChain with_structured_output(). Parse messy LLM outputs into clean Python objects.
Wed	Tool Calling Basics	Read OpenAI function calling docs and Groq tool calling reference. Build first tool-calling agent: weather lookup + calculator.	Build a 5-tool agent: search_db, calculate, format_date, convert_currency, summarize_text. Test each tool independently then together.
Thu	External APIs as Tools	Integrate real external APIs as tools: public REST APIs (weather, news, currency). Handle API errors inside tool implementations.	Build: News research agent that searches news, extracts entities, classifies topics, and summarizes findings using multiple API tools.
Fri	Integration + Demo	Build a production-grade tool-calling agent with error handling, logging, retries, and structured output. Add Streamlit UI.	Weekly standup. Demo the multi-tool agent. Code review. Architecture discussion: when to use tools vs chains?

Recommended Resources

These resources are drawn from official documentation and original research. Complete the highlighted readings before their corresponding daily session.

Type	Resource	Why It Matters
Paper	Chain-of-Thought Prompting Elicits Reasoning in LLMs (Wei et al., 2022)	Foundation of CoT techniques
Paper	Tree of Thoughts: Deliberate Problem Solving with LLMs	Advanced reasoning pattern
Docs	Anthropic Prompt Engineering Guide	Best practices from Claude's creators
Docs	OpenAI Function Calling Guide	Standard tool calling specification
Docs	LangChain Tools Documentation	How to build and bind tools
Docs	Pydantic v2 Documentation	Structured output validation
Blog	Groq Tool Calling Cookbook	Groq-specific tool calling patterns
Video	DeepLearning.AI – Building with LLM APIs (short course)	Practical function calling tutorial
Repo	LangChain Tool Examples (GitHub)	Real tool implementations
Blog	Prompt Engineering Guide (promptingguide.ai)	Comprehensive prompting reference

Hands-on Labs

Complete all three labs over the course of the week. Each lab builds toward the skills required for your Friday project submission.

Lab 2.1: Structured Output Extractor

Build a tool that takes unstructured job postings and extracts: title, company, salary range, required skills, location, and remote status , all as a validated Pydantic model.

Lab 2.2: Multi-Tool Research Agent

Build an agent with 5+ tools: web_search_mock, calculate, get_current_date, summarize, classify_sentiment. The agent must correctly route queries to the right tool.

Lab 2.3: Error Recovery Agent

Build an agent that handles tool failures: if a tool fails, it tries an alternative. Log all attempts and outcomes. Implement exponential backoff for rate limit errors.

Weekly Assessment

Complete this self-assessment before Friday's standup. These questions test conceptual clarity, not memorisation.

Question Type	Question
Conceptual	What is the difference between Chain-of-Thought and Tree-of-Thought prompting? When would you use each?
Conceptual	Why does structured output matter for production AI systems? What problems does it solve?
Conceptual	Explain the tool-calling lifecycle: how does a model decide to call a tool, and what happens after?
Technical	Write a Pydantic model for extracting a job posting's title, salary range, and required skills.
Technical	What is exponential backoff and why is it used when calling external APIs?
Design	Design a tool schema for a 'send_email' function. What parameters, types, and descriptions would you include?

Weekly Standup Requirements

Every Friday standup follows the same structure. Prepare each item in advance.

Item	Detail
Demo	Live demo of your multi-tool agent (2 minutes). Show at least 3 distinct tools being called correctly.
Technical Question	Explain one tool-calling or structured-output concept from the week that surprised you.
Architecture Review	Draw your agent's tool-routing logic: how does it decide which tool to call?
Code Review	Share your most interesting Pydantic model or error-handling pattern from the week.
Blockers	Name one tool integration or structured output challenge you faced. How did you solve it (or plan to)?

Week 2 Projects

Choose exactly one Intermediate project and one Production project to complete by Friday. Both are submitted via GitHub and demonstrated in the Friday standup.

Intermediate Projects , Choose One

Project 2-I-A: Smart Form Filler

Smart Form Filler	
Difficulty	Intermediate
Description	Build an AI agent that reads a natural language description and fills out a structured JSON form. Handles ambiguous input by asking clarifying questions. Validates output against a Pydantic schema.
Architecture	NL Input → Intent Classifier → Clarification Agent (if needed) → Form Filler → Pydantic Validator → JSON Output
Skills Learned	Structured output, Pydantic validation, clarification loops, form design
Deliverables	CLI tool, 10 test cases with expected outputs, validation report
Evaluation Criteria	90%+ accuracy on test cases. Handles ambiguous inputs. Clear clarification questions.

Project 2-I-B: Code Review Agent

Code Review Agent	
Difficulty	Intermediate
Description	Build an agent that reviews Python code and produces structured feedback: bugs found, security issues, style violations, performance suggestions. Output is a structured report with severity levels.
Architecture	Code Input → Multi-pass Reviewer (bugs, security, style, performance) → Structured Report → Markdown Output

Skills Learned	Multi-pass prompting, structured outputs, code analysis, prompt templates
Deliverables	Working agent, 5 example reviews on real Python code, comparison with pylint output
Evaluation Criteria	Identifies real bugs. Output is consistently structured. Severity ratings are reasonable.

Project 2-I-C: API Aggregator Agent

API Aggregator Agent	
Difficulty	Intermediate
Description	Build an agent that pulls data from 3 different public APIs (e.g., weather, news, stock prices) and synthesizes a morning briefing report. Uses tool calling to fetch each data source.
Architecture	Scheduler → 3 Tool Calls (parallel) → Data Aggregator → LLM Synthesizer → Formatted Report (HTML/Markdown)
Skills Learned	Parallel tool calls, async Python, API integration, report generation
Deliverables	Working agent, sample reports, README with setup instructions
Evaluation Criteria	Reports are coherent and well-formatted. Tools are modular and reusable. Error handling is robust.

Production Projects , Choose One

Project 2-P-A: Intelligent Document Processing Pipeline

Intelligent Document Processing Pipeline	
Difficulty	High
Description	Build a pipeline that processes uploaded documents (PDF, DOCX, TXT) and extracts structured information: entities, dates, key terms, summaries, action items , all as validated Pydantic models exposed via FastAPI.
Architecture	File Upload (FastAPI) → Document Parser (PyMuPDF/python-docx) → Multi-Tool Extraction Agent → Pydantic Validation → Database (SQLite) → REST API → Simple Frontend
Skills Learned	FastAPI, file processing, structured extraction, async pipelines, database integration
Deliverables	Docker-deployed FastAPI app, API documentation, demo processing 5 real documents, GitHub repo with CI
Evaluation Criteria	Handles all 3 file formats. Extraction accuracy demonstrated on test set. Docker compose works. API is documented.

Project 2-P-B: AI Function-as-a-Service Platform

AI Function-as-a-Service Platform	
Difficulty	High
Description	Build a platform where users can define custom AI tools via YAML config files and call them via a REST API. Tools are dynamically loaded, validated, and executed against the Groq backend.
Architecture	YAML Tool Registry → Dynamic Tool Loader → Tool Validation (Pydantic) → FastAPI Router → Groq Agent Executor → Response Formatter → API Client
Skills Learned	Dynamic configuration, plugin architecture, FastAPI advanced patterns, YAML parsing, tool registry pattern

Deliverables	Platform with 10 pre-built tools, YAML spec documentation, API playground UI, GitHub Actions deployment
Evaluation Criteria	New tools can be added via YAML without code changes. 10 working tools demonstrated. Architecture is extensible.

Project 2-P-C: Automated Data Analysis Agent

Automated Data Analysis Agent	
Difficulty	High
Description	Build an agent that accepts a CSV file and a natural language question, writes and executes Python pandas code to answer it, and produces a report with visualizations, data table, and narrative explanation.
Architecture	CSV Upload → Schema Analyzer → Question Parser → Code Generator (Groq) → Code Executor (sandboxed subprocess) → Visualization Generator → Report Builder → Streamlit UI
Skills Learned	Code generation, safe code execution, pandas, matplotlib, Streamlit, result interpretation
Deliverables	Streamlit app on Streamlit Cloud, 5 example datasets with questions and answers, security analysis of code execution approach
Evaluation Criteria	Correctly answers 80%+ of data questions. Visualizations are appropriate. Code execution is sandboxed. Deployed and accessible.

Week 2 Completion Checklist

Confirm every item below before Friday's standup.

	Checklist Item
☐	All 5 daily learning sessions completed (Mon–Fri)
☐	Lab 2.1 , Structured output extractor built and validated against Pydantic schema
☐	Lab 2.2 , Multi-tool research agent built with 5+ working tools
☐	Lab 2.3 , Error recovery agent implemented with exponential backoff
☐	Weekly Assessment questions answered in writing
☐	One Intermediate project chosen, built, and pushed to GitHub
☐	One Production project chosen, built, and pushed to GitHub
☐	Both projects have complete README files with setup instructions
☐	Tool-routing architecture diagram prepared for Friday's standup review
☐	Demo rehearsed and ready to present in under 2 minutes